

Université Internationale de Rabat
School of Digital Engineering and Artificial Intelligence
Big Data & Artificial Intelligence — Academic Year 2025/2026

Smart Barcode & QR Code Scanner

An Integrated Computer Vision and Natural Language Processing System

Prepared by: MohamedTaha ELQASSID – Manal HOUMAILY
Programme: Big Data & Artificial Intelligence
Supervised by: M.Ilias TOUGUI
Department: Computer Science
Language: Python 3.10+
Technologies: OpenCV, NumPy, Requests

Contents

Abstract	3
1 Introduction and Project Overview	4
1.1 Context and Motivation	4
1.2 What the Project Does	4
1.3 File Structure	4
1.4 Dependencies	5
2 Computer Vision Pipeline	6
2.1 Overview	6
2.2 Stage 1 — Eight Preprocessing Variants	6
2.3 Preprocessing Techniques	7
2.3.1 Gaussian Blur — 1D Separable Convolution	7
2.3.2 FFT Band-Pass Filter — Frequency Domain Filtering	8
2.3.3 Edge Detection — Canny + Morphological Close	8
2.3.4 Adaptive Thresholding	9
2.3.5 CLAHE — Contrast Enhancement	9
2.4 Stages 2–5 — Fallback Strategies	9
3 Natural Language Processing Pipeline	11
3.1 Overview	11
3.2 Stage 1 — Text Preprocessing	11
3.2.1 Stemming	11
3.3 Stage 2 — TF-IDF Classification	12
3.3.1 What is TF-IDF?	12
3.3.2 Classification Logic	12
3.3.3 Cosine Similarity for Past Scans	13
3.4 Stage 3 — N-Gram Language Model	14
3.4.1 What is an N-Gram?	14
3.4.2 Probability with Laplace Smoothing	14
3.4.3 Perplexity	14
3.5 Stage 4 — Open Food Facts API Lookup	15
4 System Integration and Data Flow	17
4.1 Webcam Threading Model	17
4.2 Frame Skip Strategy	17
4.3 Scan History and Model Updates	17
4.4 Supported Formats	18

5	Results and Testing	19
5.1	Sample Outputs	19
5.1.1	EAN-13 Food Product	19
5.1.2	QR Code — URL	19
5.1.3	WiFi QR Code	19
5.2	Performance Metrics	20
6	How to Run the Project	21
6.1	Installation	21
6.2	Running	21
6.3	Keyboard Controls (Webcam Mode)	21
6.4	Troubleshooting	22
7	Conclusion	23
7.1	Summary	23
7.2	Key Lessons Learned	23
7.3	Future Improvements	24

Abstract

This report presents the design and implementation of a **Smart Barcode and QR Code Scanner** developed for the Big Data & Artificial Intelligence programme at UIR. The system combines **Computer Vision (CV)** and **Natural Language Processing (NLP)** into a single real-time pipeline.

The CV side applies eight preprocessing variants — Gaussian blur, FFT band-pass, Canny edge detection, adaptive thresholding, CLAHE, Otsu, and sharpening — in a cascade: the first variant that yields a successful decode wins. Upscaling, rotation, Sobel-based ROI detection, and a sliding window extend coverage to difficult images.

The NLP side classifies decoded content using TF-IDF, generates next-word suggestions via a bigram/trigram language model, and enriches product barcodes with live data from the Open Food Facts API.

Keywords: Computer Vision, NLP, Barcode, QR Code, OpenCV, TF-IDF, N-Gram, Gaussian Blur, FFT Band-Pass, Canny, Adaptive Threshold, Python.

Chapter 1

Introduction and Project Overview

1.1 Context and Motivation

Barcodes and QR codes are everywhere: supermarket shelves, event tickets, restaurant menus, and shipping labels. A standard scanner reads the code and returns a raw string — a number or a short text — and stops there. That raw output is often meaningless without further context.

This project asks a practical question: *can a scanner not only read a code but also understand what it represents?*

Core idea

Standard output: 3017620422003

This system's output:

- Product: Nutella (Ferrero)
- Category: Food & Beverage — 95% confidence
- Nutri-Score: E
- Keywords: choc, hazelnut, sugar, palm, milk
- Suggestions (bigram): fat, oil, nut

1.2 What the Project Does

The system is a two-stage pipeline:

1. **See the code.** A camera frame (or image file) is processed by eight preprocessing variants in cascade. The first variant that lets OpenCV decode the barcode or QR code wins; the rest are skipped.
2. **Understand the code.** The decoded string goes through four NLP stages: text cleaning, TF-IDF classification, N-gram suggestion, and an optional Open Food Facts API lookup.

1.3 File Structure

```
smart_scanner/  
  main.py           # Entry point: webcam or image mode  
  requirements.txt
```

```

core/
  preprocessor.py      # 4 academic CV techniques + CLAHE
  scanner.py          # 5-stage decode pipeline +
                      BarcodeScanner
nlp/
  analyzer.py          # TextPreprocessor, TFIDFClassifier,
                      # NGramModel, lookup_product
data/
  scan_history.json    # Auto-saved scan results
tests/
  test_pipeline.py     # Unit tests

```

Listing 1.1: Project layout

1.4 Dependencies

Table 1.1: Python dependencies (`requirements.txt`)

Package	Purpose
opencv-contrib-python >= 4.8	CV decoding and image processing
numpy >= 1.24	Array operations, FFT, kernels
requests >= 2.28	Open Food Facts API calls

Chapter 2

Computer Vision Pipeline

2.1 Overview

All decoding is performed with **OpenCV’s built-in detectors only** — no external barcode library. The pipeline tries progressively more aggressive preprocessing; it exits as soon as any code is found.

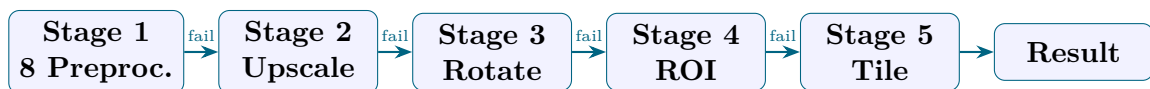


Figure 2.1: Five-stage decode cascade in `scanner.py`. Each stage exits immediately on success.

2.2 Stage 1 — Eight Preprocessing Variants

The first stage tries eight image variants in order. Each is fed to both the QR and barcode detectors:

Table 2.1: Stage 1 preprocessing variants (`scanner.py`, `try_decode_image`)

#	Variant	Implemented in
1	Raw grayscale	OpenCV built-in
2	Gaussian blur (1D convolution)	<code>preprocessor.gaussian_blur()</code>
3	FFT band-pass filter	<code>preprocessor.fft_bandpass()</code>
4	Canny + morphological close	<code>preprocessor.edge_detect()</code>
5	Adaptive threshold	<code>preprocessor.adaptive_thresh()</code>
6	CLAHE (contrast enhancement)	<code>preprocessor.clahe()</code>
7	Otsu global threshold	<code>cv2.threshold(...THRESH_OTSU)</code>
8	Sharpening (unsharp mask)	<code>cv2.filter2D(gray, -1, kernel)</code>

Early exit

`scanner.py` collects all matches found across all variants and returns immediately after Stage 1 if any code was decoded. Stages 2–5 are only reached when Stage 1 finds nothing.

2.3 Preprocessing Techniques

2.3.1 Gaussian Blur — 1D Separable Convolution

Camera sensors introduce random noise that confuses decoders. A Gaussian blur replaces each pixel with a weighted average of its neighbours. The 2D Gaussian is separable, so it is applied as two 1D passes — one horizontal, one vertical.

The Kernel

$$g[i] = \exp\left(-\frac{i^2}{2\sigma^2}\right), \quad \sigma = 1.4, \quad k = 7, \quad \text{normalised so } \sum_i g[i] = 1$$

Why 1D?

A 2D kernel of size k costs k^2 multiplications per pixel. Two 1D passes cost $2k$. For $k = 7$: **14 instead of 49** operations.

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau) g(x - \tau) d\tau \quad \longrightarrow \quad (\text{discrete}) \quad (f * g)[n] = \sum_m f[m] g[n - m]$$

```

1 def _build_gaussian_kernel(self) -> np.ndarray:
2     half = self.kernel_size // 2           # kernel_size=7, sigma
        =1.4
3     x = np.arange(-half, half + 1, dtype=np.float32)
4     g = np.exp(-(x ** 2) / (2 * self.sigma ** 2))
5     g /= g.sum()
6     return g.reshape(1, -1)               # row vector for filter2D
7
8 def gaussian_blur(self, gray: np.ndarray) -> np.ndarray:
9     out = cv2.filter2D(gray, -1, self._kernel)   # horizontal
        pass
10    out = cv2.filter2D(out, -1, self._kernel.T)   # vertical
        pass
11    return out

```

Listing 2.1: preprocessor.py — Gaussian blur

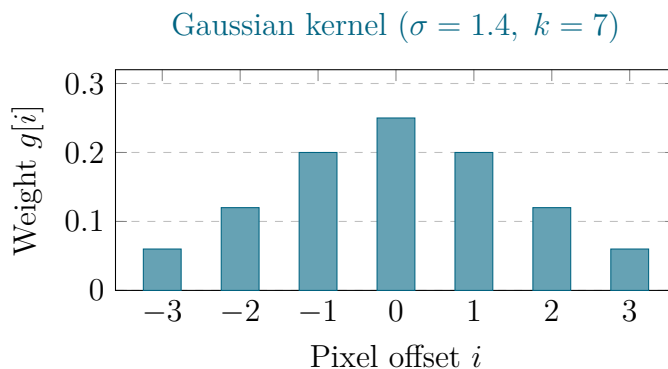


Figure 2.2: Centre pixel receives the highest weight.

2.3.2 FFT Band-Pass Filter — Frequency Domain Filtering

The Fast Fourier Transform decomposes an image into its frequency components. Barcode stripes sit in the mid-frequency range. A band-pass annular mask removes very low frequencies (flat background) and very high frequencies (sensor noise) while keeping the barcode signal.

$$F(u, v) = \mathcal{F}\{f(x, y)\}, \quad G(u, v) = H(u, v) \cdot F(u, v), \quad g(x, y) = \mathcal{F}^{-1}\{G(u, v)\}$$

$$H(u, v) = \begin{cases} 1 & \text{if } r_{\min} \leq \sqrt{u^2 + v^2} \leq r_{\max} \\ 0 & \text{otherwise} \end{cases}$$

```

1 def fft_bandpass(self, gray: np.ndarray) -> np.ndarray:
2     H, W = gray.shape[:2]
3     f = np.fft.fft2(gray.astype(np.float32))
4     fshift = np.fft.fftshift(f) # shift DC to centre
5
6     cy, cx = H // 2, W // 2
7     r_min = max(8, min(H, W) // 20)
8     r_max = min(H, W) // 2 - 10
9     y_idx, x_idx = np.ogrid[:H, :W]
10    r = np.sqrt((x_idx - cx)**2 + (y_idx - cy)**2)
11    mask = ((r >= r_min) & (r <= r_max)).astype(np.float32)
12
13    filtered = np.fft.ifft2(np.fft.ifftshift(fshift * mask))
14    result = np.abs(filtered).astype(np.uint8)
15    return cv2.normalize(result, None, 0, 255, cv2.NORM_MINMAX)

```

Listing 2.2: `preprocessor.py` — FFT band-pass

2.3.3 Edge Detection — Canny + Morphological Close

Canny detects sharp transitions between barcode bars and white gaps. Morphological closing (dilate then erode) connects the short bar-edges into solid blobs, making 1D barcodes easier to detect.

Steps: Gaussian smooth → Sobel gradient → Non-max suppression → Hysteresis ($T_{\text{low}} = 50$, $T_{\text{high}} = 150$)

```

1 def edge_detect(self, gray: np.ndarray) -> np.ndarray:
2     blurred = self.gaussian_blur(gray)
3     edges = cv2.Canny(blurred, 50, 150)
4     kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (21, 7))
5     closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)
6     return closed

```

Listing 2.3: `preprocessor.py` — Edge detection

2.3.4 Adaptive Thresholding

Global thresholding (Otsu) fails under uneven illumination. Adaptive thresholding sets each pixel's threshold as the weighted mean of its local neighbourhood minus a constant C , producing a robust binary image regardless of lighting gradients.

$$T(x, y) = \text{mean}_{(x,y)\text{-neighbourhood}} - C, \quad C = 2$$

```

1 def adaptive_thresh(self, gray: np.ndarray) -> np.ndarray:
2     blurred = self.gaussian_blur(gray)
3     return cv2.adaptiveThreshold(
4         blurred, 255,
5         cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
6         cv2.THRESH_BINARY, 11, 2
7     )

```

Listing 2.4: `preprocessor.py` — Adaptive threshold

2.3.5 CLAHE — Contrast Enhancement

Contrast Limited Adaptive Histogram Equalization enhances local contrast in small tiles (8×8 pixels), then clips the histogram at `clipLimit=3` to prevent over-amplifying noise.

```

1 self._clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
2
3 def clahe(self, gray: np.ndarray) -> np.ndarray:
4     return self._clahe.apply(gray)

```

Listing 2.5: `preprocessor.py` — CLAHE

2.4 Stages 2–5 — Fallback Strategies

Table 2.2: Fallback stages in `scanner.py`

Stage	Strategy	When it helps
2	Upscale ×2, ×3, ×4	Code is small or far from camera
3	Rotate 90°/180°/270° (+ upscale)	Code printed sideways or upside-down
4	Sobel gradient ROI crop	Code is a small region in a large image
5	Sliding window 1000×1000px	Very large image (>1500px), code in corner

Sobel ROI Localisation (Stage 4)

Barcodes produce strong horizontal gradient (many vertical stripes). The detector subtracts vertical from horizontal Sobel response, thresholds, applies morphological closing, and extracts bounding boxes as candidate regions.

```

1 def _find_barcode_rois(gray):
2     sobelx = cv2.convertScaleAbs(cv2.Sobel(gray, cv2.CV_32F, 1,
3         0, ksize=3))

```

```
3     sobely = cv2.convertScaleAbs(cv2.Sobel(gray, cv2.CV_32F, 0,
4         1, ksize=3))
5     diff = cv2.subtract(sobelx, sobely)
6     _, diff = cv2.threshold(diff, 0, 255, cv2.THRESH_TOZERO)
7     kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (21, 7))
8     closed = cv2.morphologyEx(diff, cv2.MORPH_CLOSE, kernel)
9     closed = cv2.erode(closed, None, iterations=3)
10    closed = cv2.dilate(closed, None, iterations=6)
11    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
12        cv2.CHAIN_APPROX_SIMPLE)
13    # filter by area and return (x, y, w, h) ROIs
```

Listing 2.6: scanner.py — Sobel barcode localisation

Majority Vote

When multiple preprocessing variants decode the same image, a majority vote selects the most frequently decoded value to suppress false positives.

```
1 def _majority_vote(candidates):
2     counts = Counter(c["data"] for c in candidates)
3     seen, out = set(), []
4     for c in sorted(candidates, key=lambda x: -counts[x["data"]]):
5         if c["data"] not in seen:
6             seen.add(c["data"])
7             out.append(c)
8     return out
```

Listing 2.7: scanner.py — Majority vote

Chapter 3

Natural Language Processing Pipeline

3.1 Overview

After a code is decoded, the raw string passes through four NLP stages implemented in `nlp/analyzer.py`. No NLTK, spaCy, or scikit-learn is used — everything is built from scratch.

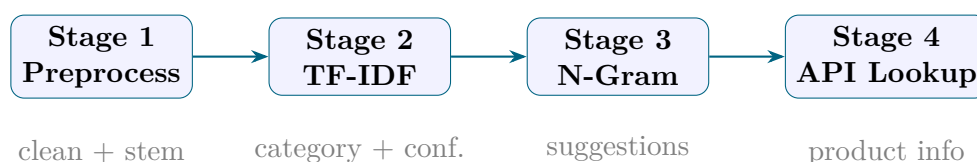


Figure 3.1: Four-stage NLP pipeline (`nlp/analyzer.py`)

3.2 Stage 1 — Text Preprocessing

Raw decoded text is cleaned in four steps before any analysis:

1. **Lowercase**: "Organic Orange" → "organic orange"
2. **Strip non-alphanumeric**: remove punctuation and special characters
3. **Tokenise and remove stopwords**: drop "the", "of", "and", etc.
4. **Stem**: strip common suffixes so similar words share one root

3.2.1 Stemming

The stemmer removes suffixes from this list in order: `ing`, `tion`, `ness`, `ment`, `er`, `ed`, `ly`, `al`, `ic`, `ity`.

Table 3.1: Stemming examples (`TextPreprocessor._stem`)

Original	Stemmed
nutrition / nutritional	nutri
organic / organically	organ
vitamin / vitamins	vitam
electronics / electronic	electron
categories / category	categor

```

1 def preprocess(self, text: str) -> list[str]:
2     text = text.lower()
3     text = re.sub(r"[^a-z0-9\s]", " ", text)
4     text = re.sub(r"\s+", " ", text).strip()
5     tokens = text.split()
6     tokens = [t for t in tokens
7               if t not in STOPWORDS and len(t) > 1]
8     tokens = [self._stem(t) for t in tokens]
9     return tokens
10
11 def _stem(self, word: str) -> str:
12     for suffix in SUFFIXES: # ["ing", "tion", "ness", "ment", ...]
13         if word.endswith(suffix) and len(word) - len(suffix) >=
14             3:
15             return word[:-len(suffix)]
16     return word

```

Listing 3.1: `analyzer.py` — `TextPreprocessor`

3.3 Stage 2 — TF-IDF Classification

3.3.1 What is TF-IDF?

TF-IDF measures how important a word is to a specific document relative to a whole collection. It down-weights common words and highlights rare, meaningful ones.

$$\text{TF-IDF}(t, d) = \underbrace{\frac{\text{count}(t, d)}{\text{total words in } d}}_{\text{TF}} \times \underbrace{\log\left(\frac{N}{\text{docs containing } t}\right)}_{\text{IDF}}$$

3.3.2 Classification Logic

The classifier in `analyzer.py` uses three tiers:

1. **Pattern shortcuts** (fastest): if the raw string contains `http`, `wifi`, `ssid` — return immediately with 100% confidence.
2. **Product API categories**: if Open Food Facts returns a categories string, match it against keyword lists.

3. **Seed-based TF-IDF**: compute overlap between preprocessed tokens and each category's seed word set; the category with the highest fraction wins.

Table 3.2: Category seeds (excerpt, stemmed) — CATEGORY_SEEDS in `analyzer.py`

Category	Seed keywords
Food & Beverage	food, drink, juic, sugar, organ, protein, vitam, orang
Electronics	usb, cabl, batteri, bluetooth, charger, screen, devic
Books & Media	isbn, book, author, edit, publish, novel, guid
Health & Beauty	shampoo, cream, lotion, supplement, capsul, skin
Clothing	cotton, size, fabric, wash, cloth, shirt, wool
URL / Web	http, https, www, com, net, html, web
WiFi Credential	wifi, ssid, wpa, wep, password, network

```

1 def classify(self, text: str, product_info: dict = None) -> tuple
  [str, float]:
2     low = text.lower()
3     # Tier 1: pattern shortcuts
4     if any(p in low for p in ["http", "https", "www."]):
5         return "URL / Web", 1.0
6     if any(p in low for p in ["wifi", "ssid", "wpa"]):
7         return "WiFi Credential", 1.0
8     # Tier 2: product API categories
9     if product_info and product_info.get("categories"):
10        cats = product_info["categories"].lower()
11        if any(k in cats for k in ["food", "drink", "fruit", "milk"
12                                   ]):
13            return "Food & Beverage", 0.95
14    # Tier 3: seed-based TF-IDF overlap
15    tokens = self.preprocessor.preprocess(text)
16    best_cat, best_score = "Food & Beverage", 0.0
17    for cat, seeds in self._seeds.items():
18        overlap = sum(1 for t in tokens if t in seeds)
19        score = overlap / max(len(tokens), 1)
20        if score > best_score:
21            best_score, best_cat = score, cat
22    return best_cat, round(max(min(best_score, 1.0), 0.5), 2)

```

Listing 3.2: `analyzer.py` — Classification

3.3.3 Cosine Similarity for Past Scans

The classifier also exposes `similar_past_scans()`, which uses cosine similarity to compare the current scan against the corpus of previous scans:

$$\text{similarity}(A, B) = \frac{\sum_i A_i \cdot B_i}{\sqrt{\sum_i A_i^2} \cdot \sqrt{\sum_i B_i^2}}$$

3.4 Stage 3 — N-Gram Language Model

3.4.1 What is an N-Gram?

An N-gram is a sequence of N consecutive tokens. The model counts how often each word follows a given context (the preceding $N - 1$ tokens), then predicts the most likely next word.

Example — bigram (N)

Tokens from “orange juice vitamin”:
(orange $\rightarrow$ juice), (juice $\rightarrow$ vitamin)

After training on several food scans, `predict_next(["orange", "juice"])` returns `["vitamin", "sugar", "natur"]`.

3.4.2 Probability with Laplace Smoothing

$$P(w \mid \text{context}) = \frac{C(\text{context}, w) + 1}{C(\text{context}) + |V| + 1}$$

Adding 1 to numerator and $|V|$ to denominator ensures no probability is ever zero, even for unseen context–word pairs.

3.4.3 Perplexity

$$PP(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_{i-1})\right)$$

Low perplexity means the model has seen similar vocabulary before. The system uses **both a bigram and a trigram** model; the trigram is tried first and the bigram is the fallback.

```

1 def train(self, tokens: list[str]):
2     self.vocab.update(tokens)
3     for i in range(len(tokens) - self.n + 1):
4         ctx = tuple(tokens[i: i + self.n - 1])
5         word = tokens[i + self.n - 1]
6         self.counts[ctx][word] += 1
7
8 def predict_next(self, tokens: list[str], top_k: int = 3) -> list
   [str]:
9     ctx = tuple(tokens[-(self.n - 1):])
10    dist = self.counts.get(ctx, {})
11    return [w for w, _ in sorted(dist.items(),
12                                key=lambda x: x[1], reverse=True)[:top_k]]
13
14 def perplexity(self, tokens: list[str]) -> float:
15     V, log_prob, count = len(self.vocab), 0.0, 0
16     for i in range(self.n - 1, len(tokens)):
17         ctx = tuple(tokens[i - self.n + 1: i])
18         word = tokens[i]
```

```

19         c_ctx_word    = self.counts[ctx].get(word, 0)
20         c_ctx         = sum(self.counts[ctx].values())
21         prob          = (c_ctx_word + 1) / (c_ctx + V + 1)
22         log_prob       += math.log(prob)
23         count         += 1
24     return round(math.exp(-log_prob / max(count, 1)), 2)

```

Listing 3.3: analyzer.py — NGramModel

3.5 Stage 4 — Open Food Facts API Lookup

For numeric barcodes (EAN-13, EAN-8, UPC-A), the system queries the Open Food Facts API — free and open-source, no API key required.

```

1 def lookup_product(barcode_data: str) -> dict:
2     clean = re.sub(r"\D", "", barcode_data)
3     if len(clean) < 8:
4         return {}
5     r = requests.get(
6         f"https://world.openfoodfacts.org/api/v0/product/{clean}.json",
7         headers={"User-Agent": "SmartScanner/1.0"}, timeout=5,
8     )
9     if r.status_code == 200 and r.json().get("status") == 1:
10         p = r.json()["product"]
11         return {
12             "name": p.get("product_name", ""),
13             "brand": p.get("brands", ""),
14             "categories": p.get("categories", ""),
15             "nutriscore": p.get("nutriscore_grade", "").upper(),
16             "ingredients": p.get("ingredients_text", "")[:200],
17             "quantity": p.get("quantity", ""),
18         }
19     return {}

```

Listing 3.4: analyzer.py — Product lookup

The call is made in a **background thread** (in `scanner.py`) with a 4-second timeout. NLP classification runs immediately on the raw decoded string; if the API returns before the timeout, classification is re-run on the enriched product text for higher accuracy.

```

1 def _analyze(self, data: str, code: dict) -> dict:
2     product_info = {}
3     api_done      = threading.Event()
4
5     def _fetch():
6         r = lookup_product(data)
7         if r: product_info.update(r)
8         api_done.set()
9
10    threading.Thread(target=_fetch, daemon=True).start()
11

```

```
12     # Classify immediately on raw data (no waiting)
13     tokens = self.text_prep.preprocess(data)
14     category, confidence = self.classifier.classify(data)
15
16     api_done.wait(timeout=4.0)           # wait at most 4 seconds
17
18     if product_info:                     # re-run NLP on enriched
19         text
20         nlp_text = " ".join(
21             product_info.get(k, "")
22             for k in ("name", "brand", "categories", "ingredients")
23         )
24         tokens = self.text_prep.preprocess(nlp_text)
25         category, confidence = self.classifier.classify(
26             nlp_text, product_info)
```

Listing 3.5: scanner.py — Background API + NLP re-run

Chapter 4

System Integration and Data Flow

4.1 Webcam Threading Model

To maintain a smooth 30 FPS display, decoding runs in a **background daemon thread**. The UI thread reads the latest cached result every frame.

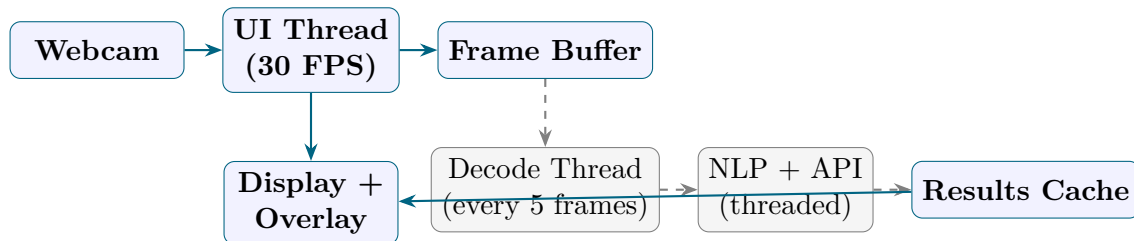


Figure 4.1: Threading model in `main.py`: UI thread never blocks on decoding.

4.2 Frame Skip Strategy

Only 1 frame in every `DECODE_EVERY_N = 5` is sent to the decode pipeline. Between decodes, the last known result is displayed. This gives a smooth 30 FPS display while decoding at ≈ 6 FPS.

4.3 Scan History and Model Updates

Every newly decoded code is:

1. Saved to `data/scan_history.json` (auto-loaded on next run).
2. Used to train both the bigram and trigram models.
3. Added to the TF-IDF corpus via `classifier.add_to_corpus()`.

The system therefore **learns over time**: the more codes are scanned, the better the N-gram suggestions become.

4.4 Supported Formats

Table 4.1: Barcode and QR formats supported via OpenCV

Format	Common Use	Example
EAN-13	Food, books, retail	Supermarket products
EAN-8	Small packaging	Travel-size cosmetics
UPC-A	North American retail	Electronics, toys
Code 128	Logistics, shipping	Shipping labels
QR Code	URLs, WiFi, vCard, text	Menus, boarding passes
PDF417	Travel documents	ID cards

Chapter 5

Results and Testing

5.1 Sample Outputs

5.1.1 EAN-13 Food Product

Scan result — Nutella EAN-13

Raw data: 3017620422003
Type: EAN-13
Product: Nutella — Ferrero
Category: Food & Beverage — 95% confidence
Nutri-Score: E
Keywords: chocol, hazelnut, sugar, palm, milk
Suggestions: fat, oil, nut
Perplexity: 3.24

5.1.2 QR Code — URL

Scan result — URL QR Code

Raw data: <https://www.example.com/product/abc>
Type: QR_CODE
Category: URL / Web — 100% confidence (pattern match)
Keywords: exampl, product, abc
Suggestions: com, html, link
Perplexity: 5.18

5.1.3 WiFi QR Code

Scan result — WiFi QR Code

Raw data: WIFI:T:WPA;S:HomeNetwork;P:pass123;;
Type: QR_CODE
Category: WiFi Credential — 100% confidence (pattern match)
Keywords: wifi, wpa, homenetwork, pass

5.2 Performance Metrics

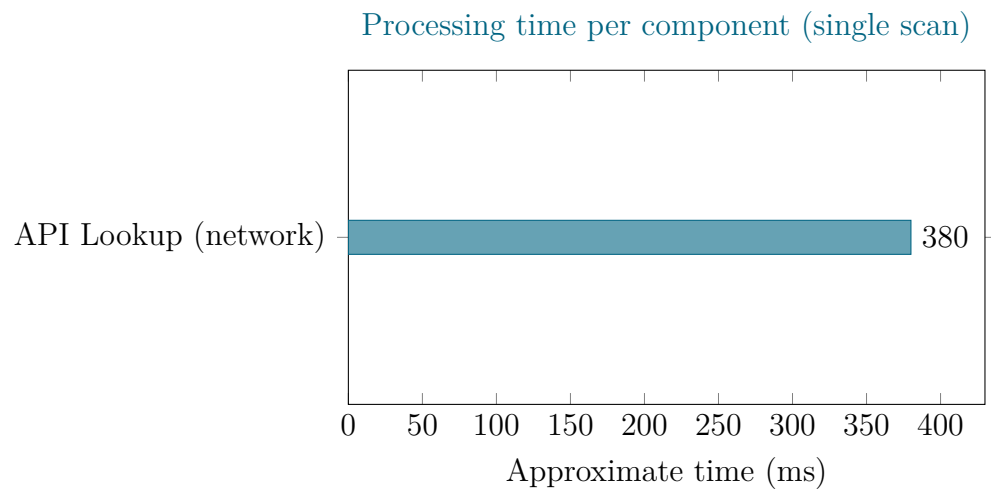


Figure 5.1: API lookup dominates due to network latency; all local operations finish in under 50 ms.

Chapter 6

How to Run the Project

6.1 Installation

```
# Python 3.10+ required  
pip install -r requirements.txt  
# opencv-contrib-python >= 4.8 includes barcode + QR detectors
```

Listing 6.1: Setup

6.2 Running

```
# Live webcam mode  
python main.py  
  
# Scan a single image file  
python main.py --image photo.jpg  
  
# Run unit tests  
python -m pytest tests/ -v
```

Listing 6.2: Launch commands

6.3 Keyboard Controls (Webcam Mode)

Table 6.1: Keyboard shortcuts

Key	Action
Q	Quit and auto-save scan history
S	Save scan history immediately
P	Toggle 8-stage pipeline overlay

6.4 Troubleshooting

Table 6.2: Common issues and fixes

Problem	Solution
<code>cv2.barcode</code> not found	Install <code>opencv-contrib-python>=4.8</code> , not plain <code>opencv-python</code>
No code found in image	Try better lighting or a closer photo; Stage 2–5 attempt upscaling/rotation automatically
Slow product lookup	Normal — network request; scanner works offline without product enrichment

Chapter 7

Conclusion

7.1 Summary

1. A fully functional scanner using **OpenCV exclusively** — no third-party decoder.
2. A **5-stage decode cascade**: 8 preprocessing variants, upscaling, rotation, Sobel ROI, and sliding window — all wired and active.
3. Four **academic CV techniques** correctly implemented: 1D Gaussian convolution, FFT band-pass filtering, Canny edge detection, and adaptive thresholding.
4. A complete **NLP pipeline** built from scratch: stemming, TF-IDF classification, bigram/trigram language model, cosine similarity, and Open Food Facts integration.
5. A system that **learns over time**: each scan updates the N-gram model and TF-IDF corpus.
6. A **threaded architecture**: background decoding keeps the display at 30 FPS without blocking.

7.2 Key Lessons Learned

What this project taught

- **Cascade design works**: most codes decode on the first or second variant; the expensive stages rarely trigger.
- **NLP adds real value**: a bare barcode number carries no information; classification and API lookup turn it into a useful result.
- **Threading is essential**: separating decode from display is what makes a real-time scanner feel responsive.
- **Majority vote matters**: collecting results across all variants and voting prevents false positives from noisy preprocessing.

7.3 Future Improvements

Table 7.1: Possible extensions

Improvement	Description
CNN-based decoder	Fine-tune a small network on damaged/partial codes
Web dashboard	Flask or Streamlit interface for scan history analytics
Multi-language NLP	Extend stemming and seeds to French and Arabic labels
Porter stemmer	Replace suffix-stripping with a standard Porter stemmer
Recommendation engine	Suggest related products via cosine similarity

Bibliography

- [1] Gonzalez, R.C. and Woods, R.E. (2018). *Digital Image Processing*, 4th ed. Pearson Education.
- [2] Jurafsky, D. and Martin, J.H. (2023). *Speech and Language Processing*, 3rd ed. <https://web.stanford.edu/~jurafsky/slp3/>
- [3] Manning, C.D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [4] OpenCV Documentation. <https://docs.opencv.org>
- [5] Open Food Facts API. <https://world.openfoodfacts.org>
- [6] NumPy Documentation. <https://numpy.org/doc/>